

Generative AI for Media Project

Francesco Lozio - 737664, Giacomo Baresi - 735878

I. ABSTRACT

In this project, we'll analyze how a synthetic image detector performs using normal images and processed images with JPEG AI, an international image coding standard based on an end-to-end learning-based approach, developed jointly by ISO, IEC, and ITU-T under the Joint Photographic Experts Group (JPEG)[2]. We explain the phenomenon that reduce accuracy on the compressed and decompressed images, especially at lower qualities. Finally, we introduce the concept of a Bridge, a Convolutional Neural Network that allows to use latents generated with JPEG AI to input them to the detector in order to obtain higher accuracy despite the low quality compression.

II. INTRODUCTION

The increasing volume of visual data generated by modern applications has created a growing demand for efficient image compression technologies. Traditional image coding standards, such as JPEG and more recent standards such as Versatile Video Coding (VVC), rely primarily on hand-designed signal-processing and coding techniques. In parallel, advances in deep learning have demonstrated that neural networks can learn efficient representations of visual data directly from large image datasets. JPEG AI was initiated to investigate the use of such learning-based techniques within an internationally standardized image coding framework.

JPEG AI is an international standard for image coding based on an end-to-end learning-based approach, jointly developed under the ISO, IEC, and ITU standardization framework through the Joint Photographic Experts Group (JPEG). The project was established in 2019, followed by a Call for Proposals in 2022. After the evaluation and integration of several learning-based coding solutions, JPEG AI reached the Draft International Standard stage in 2024, and its first core coding system was published as ISO/IEC 6048-1:2025 and ITU-T T.840.1 in 2025.

At the core of JPEG AI is an end-to-end learned image compression architecture and the main objective of JPEG AI is to provide a single-stream and compact compressed-domain representation that can serve both human and machine-oriented applications. In addition to reconstructing images for human visualization, the representation is designed to facilitate image processing and computer vision tasks directly in the compressed domain. This represents an important difference from conventional image codecs, where compressed data generally needs to be fully decoded before it can be processed by a machine-learning application.

On the other hand, detecting if an image is real or not is becoming more and more important with the diffusion of Generative AI.

However, a real image that has been compressed and decompressed with JPEG AI is more difficult to recognize as real. In this project, we'll obtain the results of detecting real and fake images using different compression quality. Lastly, we introduce the Bridge, a Convolutional Neural Network that will allow us to use latent extracted during JPEG AI compression to input them to the detector in order to increase the accuracy of the lowest quality.

III. JPEG AI

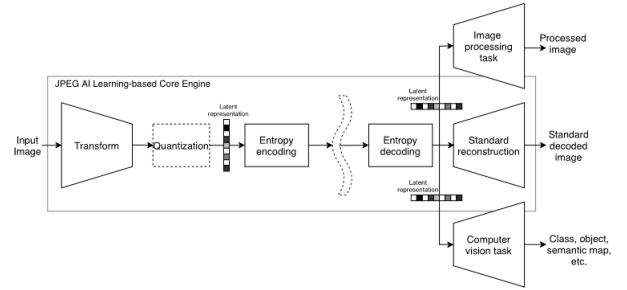


Fig. 1. JPEG AI Structure

JPEG AI provides a standard which focuses on using neural networks for learning an optimal way to compress and decompress images. Different qualities are also available: an higher quality requires more space but provides an higher resolution.

This way the compression is of higher quality than standard compressions and consumes less resources, which makes it an extremely valid framework.

The problem is with fake and real images detection: after being decompressed, the images are "dirty" and detectors struggle in correctly classify them. For this reason, in the following section we'll show the used detector and the results obtained with images compressed at different qualities.

IV. DETECTOR

The detector used in this project is the one presented in [4] and made publicly available in the corresponding repository [5]. It is a patch-based classifier: given an input image, the detector extracts 96×96 patches, where each patch represents a sub-region of the original image with the specified dimensions. Each patch is then processed by a fine-tuned *EfficientNet-B4*, whose final layer produces two scores, corresponding to the real and fake classes, respectively. The first score represents the likelihood of the patch being real, while the second represents its likelihood of being fake.

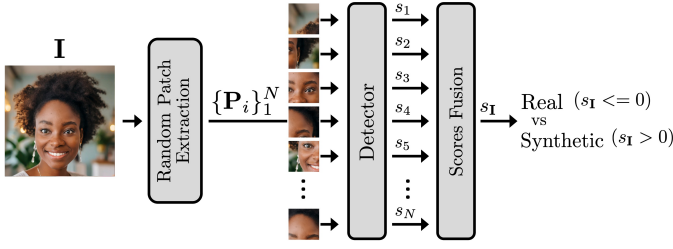


Fig. 2. Detector Working Method, from the github repository[5]

After all patches have been classified, only the top M patches are selected. A patch is considered a “top patch” if it has one of the highest fake scores among all the extracted patches. The final image-level score is then computed as the average of the fake scores of the selected M patches. Therefore, the final score represents the estimated likelihood that the image is fake, based on the patches considered most indicative of manipulation. The image is classified as fake when the resulting score is greater than 0, and as real otherwise.

Additionally, the detector can be used in combination with *BlazeFace* (BF). *BlazeFace* is a lightweight face-detection model that allows the classifier to selectively extract patches from regions containing detected human faces. This allows the detector to focus its analysis on facial regions, which are particularly relevant for face-manipulation detection.

The detector was evaluated on three different datasets, each of which was tested both with and without the *BlazeFace* component, with the exception of the Original Dataset. The datasets are described below:

- 1) OpenForensics (OF)[3]: This dataset was constructed to provide a challenging benchmark for existing forgery detectors. It contains manipulated images with different levels of difficulty, with a particular focus on face-swapping manipulations. The dataset is available at <https://zenodo.org/records/5528418>;
- 2) Deepfake Image Detection (DID)[1]: A Kaggle dataset containing both synthetic and real images. Unlike the OF dataset, this one also contains images without human subjects;
- 3) Original Dataset: The original dataset used to train the detector. Given the high accuracy (see I), no *Blaze Face* detector has been used. Refer to [4] for how the dataset has been built.

Dataset	BF	Original	Q012	Q025	Q050	Q075	Q100
Original Dataset	No	100.00%	50.00%	65.00%	84.00%	93.00%	100.00%
OF	No	38.50%	68.50%	65.50%	60.00%	60.00%	51.00%
OF	Yes	54.00%	68.50%	66.50%	64.00%	61.00%	60.00%
DID	No	68.54%	33.59%	56.25%	53.23%	53.23%	58.06%
DID	Yes	67.04%	35.16%	57.81%	53.23%	53.23%	56.45%

TABLE I

DETECTION ACCURACY ACROSS DIFFERENT JPEG AI QUALITY LEVELS AND DATASETS.

The results show substantially different behaviors across the three datasets. As expected, the highest accuracy is obtained on

the Original Dataset, where the detector achieves 100.00% accuracy on both the original images and the images encoded at quality 100. However, the accuracy decreases considerably at lower JPEG AI quality levels, reaching 50.00% at quality 12. This behavior is consistent with the expectation that stronger compression may remove or alter image characteristics used by the detector to identify manipulated content.

The results obtained on OpenForensics exhibit a different trend. The detector achieves an accuracy of only 38.50% on the original images without *BlazeFace*, while the accuracy increases to 68.50% at quality 12. This is opposite to the behavior observed on the Original Dataset, where decreasing image quality leads to a reduction in accuracy. Adding *BlazeFace* improves the accuracy on the original images from 38.50% to 54.00%, but it does not substantially change the overall trend across the different quality levels.

The DID dataset also produces lower-than-desired accuracy on the original images. Without *BlazeFace*, the detector achieves 68.54% accuracy, while the addition of *BlazeFace* slightly reduces it to 67.04%. At lower JPEG AI quality levels, the accuracy decreases further, reaching 33.59% and 35.16% at quality 12 without and with *BlazeFace*, respectively. Therefore, the addition of *BlazeFace* is not sufficient to obtain the expected performance on this dataset.

Overall, these results indicate that the detector’s behavior is highly dependent on the dataset and on the characteristics of the images being evaluated. While the Original Dataset exhibits the expected degradation in detection accuracy as JPEG AI compression becomes stronger, the OpenForensics dataset shows the opposite trend, and the DID dataset produces relatively low accuracy even on the original images.

Given these results, we choose to use the Original Dataset also for training the bridge in the next section

V. BRIDGE

From now on, we consider the lowest quality level, Q012. The goal is to improve the detector’s results. It is known that the issue with the compressed images lies in the decompression stage, which introduces synthetic artifacts into the real images. For this reason, the idea is to avoid reconstructing the images and instead use the latents extracted during compression for classification.

Latents cannot be used directly as input to the detector because their input size and number of channels do not match those expected by the detector. Therefore, a bridge is required to pass the latent directly into the detector’s *EfficientNet*, with only minor adjustments.

One major limitation was maintaining coherence with the original detector: this fine-tuned *EfficientNet* assumes an input size of 96×96 , so we had to find a block whose expected input size would be sufficiently reasonable.

A. Latent Size Considerations

In order to find this ideal injection point, we first analyzed two aspects:

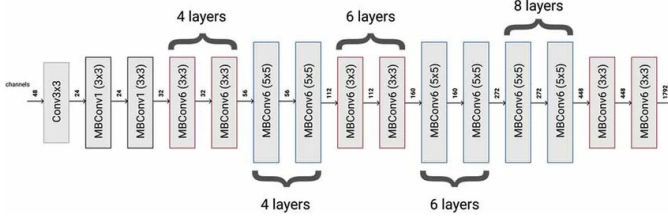


Fig. 3. EfficientNet-B4 architecture.

- **Latent Size:** In our initial experiments, all latents were $64 \times 64@256$, so we assumed that all other latents would use the same dimensions. To do so, we interpolated any latent with a different spatial size to match this expected size;
- **Patching:** When considering how much of the original detector should be preserved, the question of whether to preserve the patching scores also arose. Since we could ignore the patching and provide the full latent at a certain block, the final decision was to create two versions of the bridge, implementing both patching and a “single variant”, which means no patching at all.

B. Injection Point

We therefore considered what the ideal patch size could be, given that the original input dimension was 96×96 , by choosing a dimension that was neither too large nor too small.

We also wanted to avoid providing the latent as input to either the first or the last blocks of the network. We decided to use a patch size of $12 \times 12@256$ because the 12th block of *EfficientNet-B4* (10th in PyTorch, due to the first block being separated and indexing starting from 0) expects an input of $12 \times 12@56$ when using the detector’s weights. The reasoning is as follows:

- **Stem Convolution (Conv3x3, stride=2):** Downsamples the initial patch.
 $96 \times 96@3 \rightarrow 48 \times 48@48$
- **Blocks 0–1 (MBConv1, stride=1):** Maintains spatial dimensions.
 $48 \times 48@48 \rightarrow 48 \times 48@24$
- **Block 2 (MBConv6, stride=2):** First block of the next stage; halves the spatial dimensions.
 $48 \times 48@24 \rightarrow 24 \times 24@32$
- **Blocks 3–5 (MBConv6, stride=1):** Maintains spatial dimensions.
 $24 \times 24@32 \rightarrow 24 \times 24@32$
- **Block 6 (MBConv6, stride=2):** First block of the 56-channel stage; halves the spatial dimensions again.
 $24 \times 24@32 \rightarrow 12 \times 12@56$
- **Blocks 7–9 (MBConv6, stride=1):** Maintains spatial dimensions and finalizes the features before our injection point.
 $12 \times 12@56 \rightarrow 12 \times 12@56$

This means that if a patch bridge expects a latent size of $12 \times 12@256$, it only needs to adjust the number of channels.

Furthermore, the injection point was considered sufficiently “in-between” in the network.

For this reason, the single bridge also uses the 12th block as its injection point. In this case, however, the bridge is not composed only of a channel adjustment, but also performs spatial downsampling.

C. Bridges Structures

The bridges structures are as follow:

Latent Bridge (Single Variant):

- **Input:** $64 \times 64@256$
- **Conv 1:** 3×3 , stride 2 $\rightarrow 32 \times 32@128$ (BN + ReLU)
- **Conv 2:** 3×3 , stride 2 $\rightarrow 16 \times 16@64$ (BN + ReLU)
- **Conv 3:** 3×3 , stride 1 $\rightarrow 16 \times 16@56$ (BN + ReLU)
- **Adaptive Pooling:** $\rightarrow 12 \times 12@56$

The single bridge aims to adapt the whole latent to the required $12 \times 12@56$. This is done by using three Convolutional Layers, each of them containing a Batch Normalization and a ReLU activation. Finally, an Adaptive Average Pooling ensures the requested size.

Latent Patch Bridge (Patching Variant):

- **Input:** $12 \times 12@256$
- **Conv 1:** 3×3 , stride 1 $\rightarrow 12 \times 12@128$ (BN + ReLU)
- **Conv 2:** 3×3 , stride 1 $\rightarrow 12 \times 12@56$ (BN + ReLU)

The patch bridge is a simpler architecture: it only has 2 Convolutional Layers (followed by Batch Normalization and ReLU) that reduce the channels without modifying the size of the input latent.

D. Results

Both bridges were trained using the same training configuration, with the parameters reported below:

- 1) Epochs: 30;
- 2) Seed: 42;
- 3) Learning Rate: $1e-4$;
- 4) Optimizer: Adam;
- 5) Loss: Cross Entropy;
- 6) Batch Size (Single Bridge only): 16;
- 7) Number of Patches (Patch Bridge only): 64;
- 8) Top Patches (Patch Bridge only): 48.

The number of patches and the top ones considered vary from the test accuracy and default ones: the detector uses as default 800 patches and selects the top 600 ones. Due to temporal reason, the training has been done using only 64 patches and considered only the top 48 ones which still required two days of training.

The used test set is the same for I but uses latents extracted during the compression of the images of quality 012.

Bridge	Overall	Real	Fake
Patch Bridge	65.00%	57.00%	73.00%
Single Bridge	72.00%	86.00%	58.00%

TABLE II

DETECTION ACCURACY OF THE PATCH BRIDGE AND SINGLE BRIDGE.

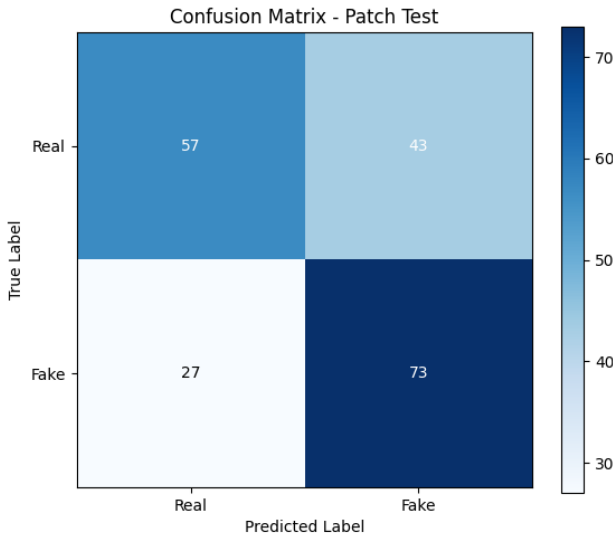


Fig. 4. Patch bridge confusion matrix

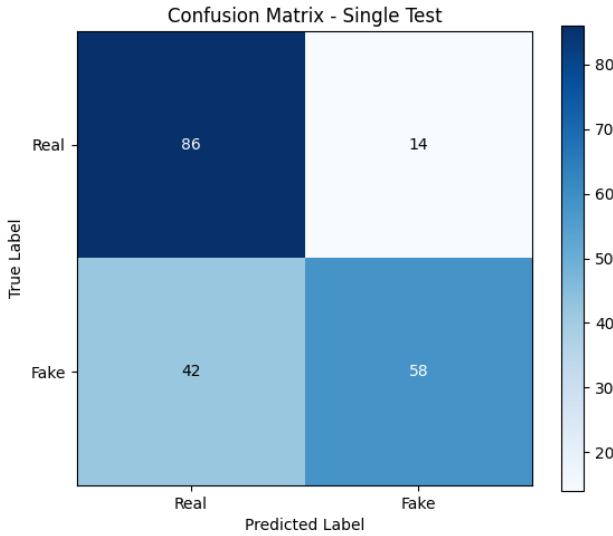


Fig. 5. Single bridge confusion matrix

The results show that both bridge architectures are able to perform classification directly from the latent representations with improved accuracy comparing to the previous quality 012 accuracy. However, the two approaches exhibit substantially different behavior depending on the class being detected.

The Patch Bridge achieves an overall accuracy of 65.00%, with an accuracy of 57.00% on real images and 73.00% on fake images. Therefore, although its overall performance is lower than that of the Single Bridge, it is more effective at identifying fake images in this experiment. In contrast, the Single Bridge achieves an overall accuracy of 72.00%, with a substantially higher accuracy on real images of 86.00%. However, its accuracy on fake images drops to 58.00%.

This difference is particularly relevant when considering the

original problem addressed by the bridge. The decompression stage was observed to introduce synthetic artifacts into real images, causing the detector to incorrectly classify some real images as fake. The Single Bridge appears to mitigate this issue more effectively, as demonstrated by its 86.00% accuracy on real images. At the same time, this improvement comes with a decrease in fake-image detection accuracy compared with the Patch Bridge.

The two architectures therefore exhibit a clear trade-off between the detection of real and fake images. The Patch Bridge favors fake-image detection, whereas the Single Bridge favors real-image detection and achieves the highest overall accuracy. Given the original objective of reducing the degradation introduced by the decompression stage, the Single Bridge provides the more promising result, despite its lower accuracy on fake images.

VI. USE OF GENERATIVE AI

As by request, this section discloses the use of Generative AI for code creation in this project. The use of Generative AI was limited to trivial and structural tasks, but for the sake of clarity, it has been documented here:

- Scripts `extract_labels.py`, `extract_random_images.py`, `prepare_dataset.py`: these scripts were used to collect images from larger datasets and label them. Specifically:
 - `extract_random_images.py` was used to handle the *OpenForensics* dataset to extract only a random subset of images;
 - `extract_labels.py` was used to handle the *OpenForensics* dataset labeling, which was provided as a JSON file. The authors understood the structure and how to retrieve the labeling, and used Generative AI to create a script that could accept a subset of such images and create a CSV associating them with their respective label using the name of the files and the provided JSON paired with corresponding instructions;
 - `prepare_dataset.py` was used to extract images from the *archive.zip* original dataset. The limitations of the Virtual Machine did not allow for extracting all the images at once (which were over 10,000). For this reason, a script for selectively extracting images without decompressing the whole folder was required. The authors analyzed the content of the compressed folder through the VM Terminal, understood the structure, and used Generative AI to handle the programmatic extraction.
- Notebook "Monkey-Patch": In attempting to improve the performance of the detector evaluation, Generative AI proposed the use of a "Monkey-Patch" to prevent a re-instantiation of the BlazeFace detector for every image by replacing an existing method of the detector with a custom one

named `patched_synth_img_detection`. While the idea was originally proposed by the AI assistant, once the authors understood the underlying concept, they implemented their own `patched_synth_img_detection` by copying the original function from the repository, finding the critical bottleneck, and replacing it with a global variable to accelerate the process;

- Bridge: In the bridge scripts (all four scripts), Generative AI handled minor structural tasks and issues such as missing imports, CSV result saving, command-line argument parsing to allow usage via Terminal, and the practical implementation of the detector injection (how to programmatically access and freeze the network's internal blocks from the outside).

VII. CONCLUSION

The proposed bridge provides a method for injecting latent representations into existing architectures, such as the detector considered in this project, allowing classification to be performed directly from the compressed representation. Despite the degradation introduced by low-quality compression, the results show that the information required for deepfake detection can still be extracted from the latents without reconstructing the images.

The two bridge variants were designed according to the structure of the original detector. For a standard detector without a patching mechanism, the Single Bridge provides a general solution for adapting the latent representation to the expected input of the network. The Patch Bridge, instead, can be used when the target architecture relies on patch-based processing, allowing the bridge to preserve this aspect of the original detector.

Although the accuracy obtained from the latent representations did not reach the performance achieved on the original images, the observed improvements demonstrate that retrieving features suitable for deepfake detection directly from the latent representation is feasible. In particular, the different behaviors of the two bridge architectures highlight the importance of adapting the bridge to the characteristics of the target detector.

The proposed bridge architectures are relatively simple and can therefore be further improved. Future work could investigate more complex architectures and alternative injection points to better adapt the latent representations to the detector while preserving more of the information required for distinguishing between real and fake images.

REFERENCES

- [1] *Deepfake Image Detection Dataset*. URL: <https://www.kaggle.com/datasets/saurabhbagchi/deepfake-image-detection>.
- [2] *JPEG AI. JPEG AI*. URL: <https://jpeg.org/jpegai/>.
- [3] Trung-Nghia Le et al. "OpenForensics: Large-Scale Challenging Dataset For Multi-Face Forgery Detection And Segmentation In-The-Wild". In: *International Conference on Computer Vision*. 2021.
- [4] Sara Mandelli, Paolo Bestagini, and Stefano Tubaro. "When Synthetic Traces Hide Real Content: Analysis of Stable Diffusion Image Laundering". In: *arXiv preprint arXiv:2407.10736* (2024).
- [5] Mandelli, Sara and Bestagini, Paolo and Tubaro, Stefano. *Used Detector from: When Synthetic Traces Hide Real Content: Analysis of Stable Diffusion Image Laundering*. URL: <https://github.com/polimi-ispl/synthetic-image-detection>.